ROUND 2 / PRODUCT BUILD

RelayOps

Evidence-led project handovers, with human review at the center.

What the product does

RelayOps turns project records into a traceable continuation plan. A lead reviews every finding against its source evidence, resolves contradictions, approves the snapshot, and shares a clearance-filtered handover that the recipient can acknowledge.

Project Chronos is the ready-to-use synthetic demo. Open the local app and the workflow is already populated: decisions, a task/repository disagreement, an ownership evidence gap, operational risks, next actions, and a lead-only note. No login or setup walkthrough is needed.

Core workflow

01 INTAKE	02 RECONSTRUCT	03 REVIEW
Open the seeded workspace or import Markdown / RelayOps JSON.	Group source observations into evidence-linked findings.	Inspect excerpts; confirm, edit, resolve, reject, or accept a gap.
04 APPROVE	**05 HAND OVER**	**06 ACKNOWLEDGE**
Approval waits for a reviewer decision on every finding.	Contributor and lead views receive role-filtered information.	Recipient records local receipt of the approved snapshot.

Run locally

Install dependencies with `npm install`, start with `npm run dev`, then open `http://localhost:3000`. The core workflow has no hosted service or API-key dependency.

01 / DELIVERY

Completed features

- **Ready-to-use project:** Project Chronos loads on first visit with synthetic source records and seeded findings.
- **Source intake:** import local Markdown and RelayOps JSON. File count, size, extension, structure, and duplicate IDs are checked; failures preserve the existing workspace.
- **Traceable reconstruction:** typed deterministic rules group observations into decisions, ownership, blockers, risks, and next actions, with source ID, kind, author, date, and excerpt.
- **Honest evidence states:** findings display VERIFIED, CONFLICT, or MISSING EVIDENCE. Contradictions show both sides; gaps are not presented as facts.
- **Human review:** confirm, edit, resolve conflict, accept a gap, reject, or reopen a finding. Approval is blocked until each finding has a reviewer disposition.
- **Role-aware handover:** lead-only claims and citations are removed from the contributor projection before handover components receive it. The recipient can acknowledge receipt.
- **Local workflow state:** review, approval, and receipt state use versioned browser localStorage; reset restores the synthetic workspace.
- **Accessible responsive UI:** status text, keyboard-accessible controls, visible focus, narrow-screen layout, and reduced-motion support.

Technology and implementation

- **UI:** Next.js 16.4.5 App Router, React 19.3.1, TypeScript 5.9.3.
- **Input:** bundled JSON fixture plus browser File API for Markdown and RelayOps JSON.
- **Processing:** typed validation, deterministic topic/value reconstruction, and role-specific projection; no LLM or external network required.
- **State:** browser-local versioned storage; no account, server database, or collaboration service.

Main data flow

```
Local fixture / imported file
        ↓ validate
Typed source records → deterministic findings
        ↓ cite + classify evidence
Human decisions → approval gate
        ↓ clearance projection
Recipient handover → receipt
```

02 / PRODUCT DECISIONS

Changes from Round 1

The Round 1 proposal included live GitHub ingestion, Supabase/PostgreSQL, vector search, LLM-assisted extraction, and orchestration. Round 2 deliberately narrows this to a local, synthetic and file-based workflow so judges can complete the central experience without credentials, hosted infrastructure, or an API key.

The evidence model, source traceability, conflict queue, human approval, role-aware handover, and recipient acknowledgement remain central. The broader integrations and LLM reasoning are deferred.

Round 3: three meaningful improvements

1. Voice exit micro-interview

Prompt a departing teammate about a specific evidence gap; transcribe their response and attach timestamped testimony as a cited source.

2. Dependency graph and Day-1 runbook

Visualize relationships between blockers and generate an interactive first-week checklist from approved dependencies and next actions.

3. Audit and export

Show source-to-human edit diffs, record recipient sign-off, and export the approved handover as a PDF.

Limits to understand

- Role selection is client-side demonstration logic, not a security boundary. Sample records are synthetic; do not enter confidential data.
- Browser-local state is per browser and device; there is no shared account or server persistence. Privacy-isolated contexts can clear local storage.
- Markdown extraction recognises documented headings and bullets. Unstructured text is retained as a source and surfaced as an evidence gap.
- Imported observations and excerpts are user-supplied. Shape validation does not independently prove that excerpts match external records.
- No live GitHub connector, LLM extraction, PDF handover export, immutable audit service, or public deployment is included in this local build.

03 / VERIFICATION

Judge-ready walkthrough

The demo can be followed from the seeded workspace without a presentation or operator assistance:

- Inspect the seven seeded findings and open their cited source excerpts.
- Resolve both sample conflicts; keep the ownership gap visibly unresolved as missing evidence.
- Review all findings and approve the handover snapshot.
- Switch to contributor view and verify the lead-only note is represented only by a generic redaction count.
- Switch to project-lead view to see the restricted context; acknowledge receipt in the incoming view.
- Import `data/examples/import-example.md`; confirm the source count and new batch-window, risk, and next-action findings update.
- Use reset-to-sample to return to the eight seeded sources and seven baseline findings.

Local checks

- `npm run typecheck` - passed.
- `npm run build` - passed; route `/` prerendered as a static page.
- Manual browser walkthrough covered conflicts, gap visibility, approval, role filtering, lead access, receipt acknowledgement, Markdown import, and reset.

Persistence note

The app stores workflow state in browser localStorage. The in-app browser used for this walkthrough did not reliably retain it across reloads, so persistence across reloads still needs confirmation in a standard browser. The submission does not claim that this check passed.

Repository guide

Source is organized into `app/`, `components/`, `lib/`, and `data/`. Setup, import formats, demo steps, and privacy limits are documented in README. The feature list, implementation summary, Round 1 changes, Round 3 commitments, and local verification notes are in `docs/round-2-submission.md`.

RelayOps - local-first Round 2 MVP - Report prepared 8 November 2026